



Practica Parciales

1er fecha 2025

Patrones

OO2 - Parcial 7/6/2025

Ejercicio 1- Envío de paquetes

Se quiere mejorar un sistema de envíos de paquetes que permite registrar los paquetes enviados y calcular sus costos.

De los paquetes se conoce: el valor declarado, la dirección de destino, el destinatario y una descripción, como puede verse en el diagrama UML. El costo de envío es el 5% del valor declarado.

Paquete
-descripcion: String
-destinatario: String
-direccionDestino: String
-valorDeclarado: Real
+getDescripcion(): String
+getDestinatario(): String
+getDireccionDestino(): String
+getValorDeclarado(): Real
+getCostoEnvio(): Real

Se plantea la incorporación de servicios adicionales para personalizar la entrega, mejorando la seguridad, velocidad en el envío o manejo especial del paquete, sin modificar la clase Paquete mencionada anteriormente, pero permitiendo que la clase pueda implementar una interfaz en caso de ser necesario.

Los servicios adicionales que se quieren implementar son:

- **Con seguro:** Incrementa el costo de envío en un 20% del valor declarado y agrega la nota "con seguro" a la descripción.
- **Con seguimiento en tiempo real:** permite que el cliente siga el trayecto en tiempo real. Esto incrementa el costo de envío en 2000 pesos, pero no modifica la descripción.
- **Entrega exprés:** Incrementa el costo de envío en un 50% del valor declarado y agrega la nota "entrega express" a la descripción.
- **Manipulación frágil:** No incrementa el costo, pero agrega la nota "frágil" a la descripción.

Se pide que los servicios puedan combinarse libremente.

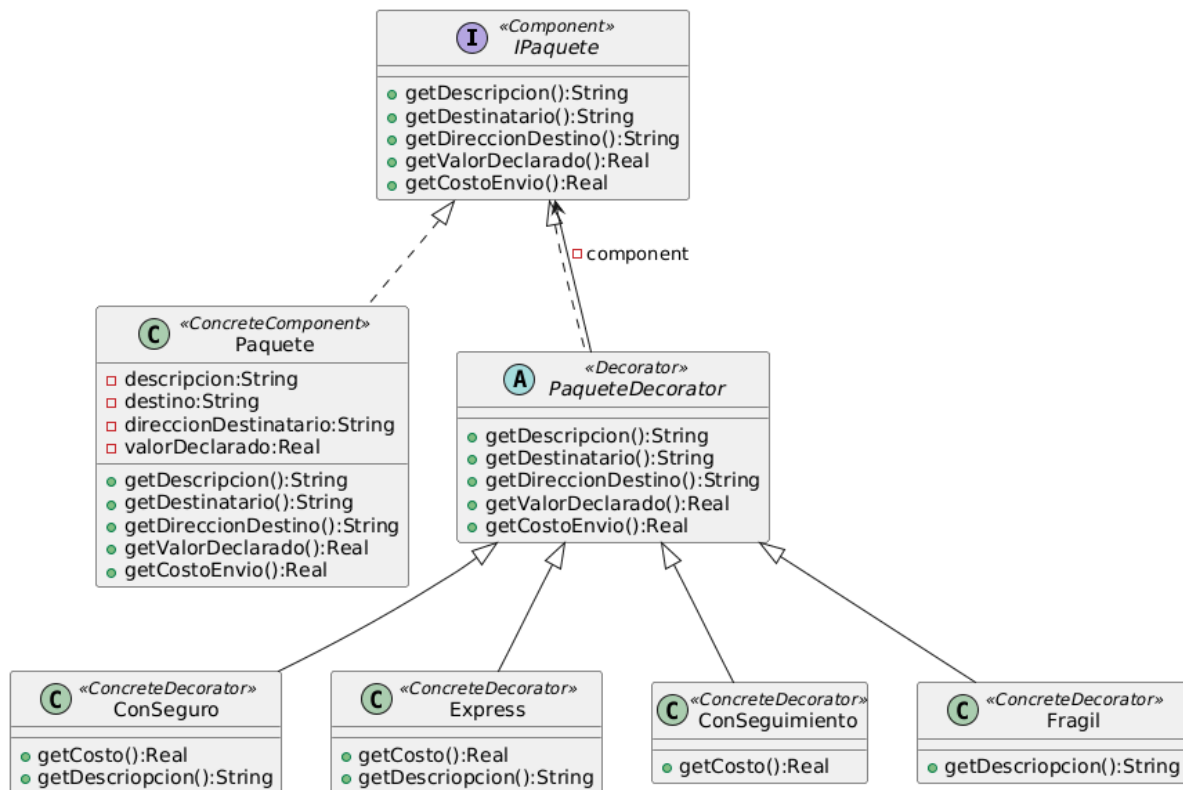
Por ejemplo, un paquete descrito como una "Caja de libros" con un valor declarado de \$20.000 que incluye los servicios adicionales de seguro (\$4.000) y entrega express (\$10.000), resulta en

- Costo de envío: 5%: \$ 1.000
- Seguro 20% de 20.000: \$ 4.000
- Entrega express 50% de 20.000: \$ 10.000
- Costo final del envío: \$ 15.000

y la descripción de "Caja de libros con seguro entrega express".

Tareas (debe realizar los cuatro ítems para aprobar el tema):

1. Realice un diagrama UML de clases para su solución al problema planteado.
2. Indique claramente en el diagrama UML el o los patrones de diseño que utiliza en el modelo y el rol que cada clase cumple en cada uno.
3. Implemente su solución en Java
4. Escriba un test que muestre cómo instanciar y utilizar su solución para el caso de la "Caja de libros" con los servicios detallados anteriormente.



```

public interface IPaquete {
    public String getDescripcion();
    public String getDestinatario();
    public String getDireccionDestino();
    public double getValorDeclarado();
    public double
  
```

```

public abstract class PaqueteDecorator implements IPaquete {
    private IPaquete component;

    public PaqueteDecorator(IPaquete c) {
        this.component = c;
    }
  
```

```

public class Express extends PaqueteDecorator {
    public Express(IPaquete component) {
        super(component);
    }

    public double getCostoEnvio() {
        return super.getCostoEnvio() +
            this.getValorDeclarado() *
  
```

```

double getCostoEnvio();
}

```

```

public class Paquete implements IPaquete{
    private String descripcion;
    private String destinatario;
    private String direccionDestino;
    private double valorDeclarado;

    public String getDescripcion(){
        return this.descripcion;
    }

    public String getDestinatario(){
        return this.destinatario;
    }

    public String getDireccionDestino(){
        return this.direccionDestino;
    }
}

```

```

    public String getDescripcion(){
        return this.component.getDescripcion();
    }

    public String getDestinatario(){
        return this.component.getDestinatario();
    }

    public String getDireccionDestino(){
        return this.component.getDireccionDestino();
    }

    public double getCostoEnvio(){
        return this.component.getCostoEnvio();
    }
}

```

```

0,5;
}

    public String getDescripcion(){
        return super.getDescripcion() + "entrega express"
    }
}

```

```

public class Fragil extends PaqueteDecorador{
    public Fragil(IPaquete component){
        super(component);
    }

    public String getDescripcion(){
        return super.getDescripcion() + "fragil"
    }
}

```



```

onDestino; }
    public double
    getValorDeclarado() {
        return this.valorDeclarado;
    }
    public double
    getCostoEnvio() {
        return this.valorDeclarado * 0.05;
    }
}

```

```

component.
getValorDeclarado();
    }
    public
    double getCostoEnvio() {
        return this.
        component.
        getCostoEnvio();
    }
}

```

```

public class ConSeguro
    extends PaqueteDecorator {
    public
    ConSeguro(IPaquete
    component) {
        super(component);
    }

    public
    double getCostoEnvio() {
        return sup

```

```
er.getCost  
oEnvio()
```

```
+ this.get  
ValorDecla  
rado() *  
0,2;  
}
```

```
public  
String get  
Descripcion()  
{  
    re  
turn supe  
r.getDescr  
ipcion() +  
"Con Segur  
o"  
}
```

```
public cla  
ss ConSegu  
imiento ex  
tends Paqu  
eteDecorat  
or{  
    public  
ConSeguimi  
ento(IPaqu  
ete compon  
ent){  
        su  
per(compon  
ent);  
    }  
    public  
double get
```

```
CostoEnvio  
( ){  
  
    return super.getCost  
oEnvio() +  
2000;  
}
```

Refactoring

OO2 - Parcial 7/6/2025

Dado el siguiente código que implementa la generación de reportes

<pre> 1 public class ReportGenerator { 2 private String type; 3 public ReportGenerator(String type) { this.type = type; } 4 5 public void generateReport(Document document) { 6 if ("PDF".equals(type)) { 7 // crear documento y configurar metadatos 8 DocumentFile docFile = new DocumentFile(); 9 docFile.setTitle(document.getTitle()); 10 String authors = ""; 11 for (String author : document.getAuthors()) { 12 authors += " " + author; 13 } 14 docFile.setAuthor(authors); 15 docFile.setContentType("application/pdf"); 16 docFile.setPageSize("A4"); 17 18 // crear exportador y setear el contenido 19 PDFExporter exporter = new PDFExporter(); 20 byte[] content = exporter.generatePDFFile(document); 21 docFile.setContent(content); 22 23 // guardar el documento 24 this.saveExportedFile(docFile); 25 } else if ("XLS".equals(type)) { 26 // crear documento y configurar metadatos 27 DocumentFile docFile = new DocumentFile(); 28 docFile.setTitle(document.getTitle()); 29 String authors = ""; 30 for (String author : document.getAuthors()) { 31 authors += " " + author; 32 } 33 docFile.setAuthor(authors); 34 docFile.setContentType("application/vnd.ms-excel"); 35 docFile.setSheetName(document.getSubTitle()); 36 37 // crear exportador y setear el contenido 38 ExcelWriter writer = new ExcelWriter(); 39 byte[] content = writer.generateExcelFile(document); 40 docFile.setContent(content); 41 42 // guardar el documento 43 this.saveExportedFile(docFile); 44 } </pre>	<pre> class ReportGeneratorTest { ReportGenerator generatorPDF; ReportGenerator generatorXLS; Document document; @BeforeEach void setUp() { document = new Document("Informe"); document.addAuthor("Carlos"); document.addAuthor("Ana"); generatorPDF = new ReportGenerator("PDF"); generatorXLS = new ReportGenerator("XLS"); } @Test void testPDF() { generatorPDF.generateReport(document); // aca se detallan los asserts } @Test void testXLS() { generatorXLS.generateReport(document); // aca se detallan los asserts } </pre>
---	---

Tareas: (debe realizar los tres ítems para aprobar el tema):

1. Enumere los code smell que encuentra en el código indicando las líneas afectadas.
2. Indique que refactorings utilizará para solucionarlos considerando que se desea incluir nuevos formatos para exportar (XLSX, CSV entre otros). Explique los pasos necesarios para realizar los refactorings elegidos, haciendo referencia al código cuando corresponda. Muestre el código final resultado luego de aplicar esos refactorings en la clase ReportGenerator y si hace falta, en la clase ReportGeneratorTest.
3. Realice el diagrama de clases del código refactorizado. Si utilizó un patrón de diseño, indíquelo en el diagrama mostrando los roles del patrón.

Bad Smell: Long Method (lineas 1 a 44)

Refactoring: Extract method

- Extraer la funcionalidad de generar un reporte dependiendo el tipo de archivo a un metodo o varios
- Extraer la funcionalidad de crear exportador y setter de contenido a un metodo abstracto que cada clase implementa a su forma

Bad Smell: Switch Statement (linea 6 a 25)

Refactoring: Replace logic conditional with Strategy

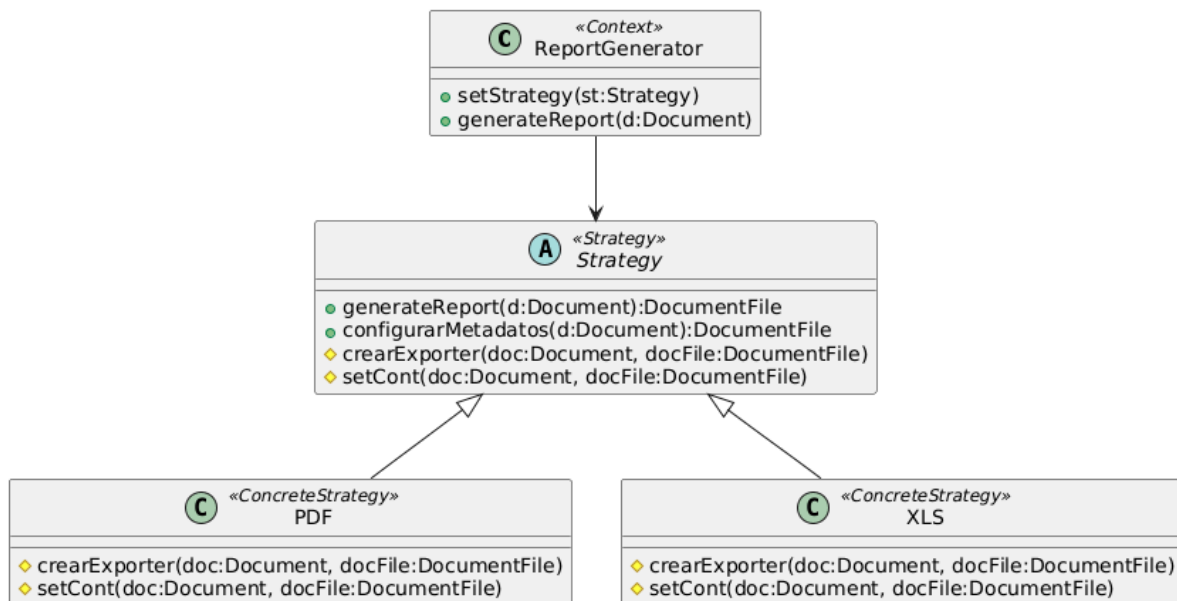
- Creo una clase strategy
- Muevo los metodos extraidos a la superclase (el de generar y el de metadatos)
- Creas metodos abstractos protegidos para la funcionalidad de la creacion del exportador y setter de contenido de pagina
- Remover el string type (en la clase ReportGenerator) que identifica el tipo de archivo y dejar que la clase misma lo represente
- Agregar un setStrategy en Strategy para poder cambiar el tipo de generador/exportador
- Bad smell: Duplicate code de creador/metadatos (lineas 8-16 y 26-35) y exportador de archivos (lineas 18-23 y 38-42)
- Refactoring: Replace conditional with Polymorphism + Extract Method. Subis metodos extraidos a la superclase Strategy con un Extract Method para dejar pasos en comun ahi

Bad smell: Comments (lineas 7,18,23,26,37, y 42)

Refactoring: Remove comments

Bad smell: Reinventando la rueda (Lineas 11-13, 30-32)

Refactoring: Replace loop with pipeline streams



▼ CLASE de TEST

```

class ReportGeneratorTest{
    ReportGenerator pdf;
    ReportGenerator xls;
    Document document;
    @BeforeEach
    void setUp(){
        Document = new Document("Reporte")
        Document.addAuthor("Pepe")
        Document.addAuthor("Carlos")
        pdf = new ReportGenerator(new PDF());
        xls = new ReportGenerator(new XLS());
    }
    @Test
    void testPDF(){
        pdf.generateReport(document)
    }
    @Test
    void testXLS(){
        xls.generateReport(document)
    }
}

```

```

public class ReportGenerator{
    private Strategy st;

    public ReportGenerator(Strategy st){
        this.st = st;
    }

    public void setStrategy(Strategy st){
        this.st = st;
    }

    public void generateReport(Document d)
    {
        d = this.st.generateReport(d)
        this.saveExportedFile(d);
    }
}

```

```

public abstract class Strategy{
    public DocumentFile generateReport(Document d){
        DocumentFile docfile = this.configurarMetadatos(d);
        this.crearExporter(d,docfile);
        return docfile;
    }
}

```

```

public class PDF extends Strategy{
    protected void crearExporter(Document doc, DocumentFile docfile){
        PDFExporter exporter = new PDFExporter();
        byte[] content = exporter.generatePDFFile(doc);
        docfile.setContent(content);
    }

    protected void setCont(Document doc, DocumentFile docfile){
        docfile.contentType("application/pdf");
        docfile.setPageSize("A4");
    }
}

```

```

public class XLS extends Strategy{
    protected void crearExporter(Document doc, DocumentFile docfile){
        ExcelWriter writer = new ExcelWriter();
        byte[] content = exporter.generateExcelFile(doc);
        docfile.setContent(content);
    }

    protected void setCont(Document doc, DocumentFile docfile){
        docfile.contentType("application/vnd.ms-excel");
        docfile.setPageSize("A4");
    }
}

```

```

    }

    public DocumentFile configurarMetadatos(Document d){
        Document docfile = new DocumentFile();
        docfile.setTitle(d.getTitle())
        docfile.setAuthor(this.d.getAuthors()
            .stream()
            .collect(Collectors.joining(", ")));

        this.setCont(d, docfile);
    }

    protected void crearExporter(Document doc, DocumentFile docfile);
    protected void setCont(Document doc, DocumentFile docfile);
}

```

```

        document doc, DocumentFile docfile){
            docfile.setContentType("application/vnd.ms-excel");
            docfile.setPageSize(doc.getSubtitle());
        }
    }
}

```


Consideremos una parte del framework `SingleThreadTCPServer` que permite construir servidores TCP en Java. El framework define el método `handleClient` que se encarga de leer los mensajes que envía el cliente, procesarlos y responder.

(A) SingleThreadTCPServer

```
+startLoop(args: String[]): void
#checkArguments(args: String[]): void
#displayUsage(): void
#displaySocketInformation(portNumber: int): void
#acceptAndDisplaySocket(serverSocket: ServerSocket): Socket
#displaySocketData(clientSocket: Socket): void
#displayAndExit(portNumber: int): void
-handleClient(clientSocket: Socket): void
+«abstract» handleMessage(message: String, out: PrintWriter): void
```

```
1 private final void handleClient(Socket clientSocket) {
2     try {
3         PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
4         BufferedReader in = new BufferedReader(
5             new InputStreamReader(clientSocket.getInputStream()));
6         String line;
7         while ((line = in.readLine()) != null) {
8             System.out.println("Received message: " + inputLine
9                 + " from " + clientSocket.getInetAddress().getHostAddress()
10                + ":" + clientSocket.getPort());
11             if (line.isEmpty()) break;
12             this.handleMessage(line, out);
13         }
14         System.out.println("Connection closed with " + clientSocket.getInetAddress().getHostAddress()
15             + ":" + clientSocket.getPort());
16     } catch (IOException e) {
17         System.err.println("Error: " + e.getMessage());
18     } finally {
19         try {
20             clientSocket.close();
21         } catch (IOException ignored) {}
22     }
23 }
24 }
25 }
```

Tareas (debe realizar los cuatro ítems para aprobar el tema):

1. Explique brevemente cuáles son los frozen spot y hot spot del código presentado del framework.
2. Modificar este framework para permitir personalizar los mensajes que se muestran en consola por defecto cuando un cliente se conecta y se desconecta
3. Explique brevemente cuáles son los frozen spot y hot spot después de la extensión realizada.
4. Indique en dónde se produce la inversión de control en la extensión realizada.

1) Los frozen spots del código se corresponden a la lógica desde la línea 1 hasta la línea 12 y de la línea 14 hasta las 25, ya que es toda la lógica ya implementada que el programador no puede ni debe modificar. Por otro lado, los hotspots del programa se encuentran en la línea 13 ya que el método `handleMessage(line, out)` permite al programador implementarlo y cambiar el comportamiento, extendiendo la clase `SingleThreadTCPServer`

2) Para permitir a este framework personalizar los mensajes de bienvenida cuando un cliente se conecta y se desconecta se agregaría un método `showMessage()` o `displayWelcomeMessage()` abstracto y que cada clase que herede de `STTCPServer` implemente a su manera el mensaje de bienvenida. Por otro lado cuando el cliente se desconecta en la línea 15 a 17, por lo que haría también un método abstracto `displayExitMessage()` con las mismas propiedades, cada clase que herede de `STTCPServer` implementaría su mensaje de despedida a su forma y listo. Con estas nuevas adiciones se podría eliminar ambos `System.out.print` de las líneas 8-10 y 15-16 respectivamente.

El código modificador quedaría:

```
// Nuevos métodos abstractos para envío de mensajes
public abstract void displayWelcomeMessage();
public abstract void displayExitMessage()

private final void handleClient(Socket clientSocket){
    try {
        PrintWriter out = new PrintWriter(clientSocket.get
```

```

OutputStream(),true);
    BufferedReader in = new BufferedReader(new InputStr
eamReader(clientSocket.getInputStream()));
    String line;

    // Estas lineas se podrian eliminar y dejar solo th
is.displayWelcomeMessage
    // while ( (line = in.readLine()) != null{
        // System.out.println("Received message" + inpu
tLine + "from" + clientSocket.getInetAddress()
        // .getHostAddress() + "." clientSocket.getPort
());
    this.displayWelcomeMessage();

    if (line.isEmpty()){ break;
        this.handleMessage(line,out);
    }
    // Estas lineas se podrian eliminar y dejar solo th
is.displayExitMessage
    // System.out.println("Connection closed with" + cl
ientSocket.getInetAddress().getHostAddress(_
    // + "." + clientSocket.getPort(0
    this.displayExitMessage();

} catch (IOException e) {
    System.err.println("Error: " + e.getMessage());
} finally {
    try {
        clientSocket.close();
    } catch (IOException ignored) {}
}
}

```

3) Los frozenspot luego de la modificacion/extension realiz
ada quedarian desde la linea
1-10, 12-16 y 18-25 ya que los hotspots nuevos estarian en
la linea 11 y 17 respectiva-
mente.

4) La inversion de control, es decir, cuando el framework toma control es en la linea 11 y 17, en la antigua version sin extension esto ocurría en la linea 13 con el metodo `handleMessage(line,out)`